

Product Report

AI-Powered Resume Analyzer & Interview Preparation Platform

1. Executive Summary

The AI-Powered Resume Analyzer & Interview Preparation Platform is a Retrieval-Augmented Generation (RAG)-based system designed to automate resume analysis, semantic skill matching, and personalized interview preparation.

The platform compares candidate resumes against target Job Descriptions (JDs) using semantic vector retrieval instead of traditional keyword-based matching. By leveraging structured document parsing, embeddings, vector databases, and Gemini-powered generation, the system identifies missing skills, generates contextual interview questions, and provides personalized preparation guidance.

The primary objective of the system is to reduce manual screening effort, improve semantic matching accuracy, and generate highly contextual career guidance instead of generic AI-generated outputs.

2. Business Problem

Traditional resume screening systems rely heavily on keyword matching.

This creates multiple limitations:

- Resumes containing semantically similar technologies are often ignored.
- Different terminology creates false negatives.
- Manual resume review is time-consuming.
- Generic AI prompts generate unreliable interview preparation.
- Standard ATS systems fail to preserve document structure.

Example:

A Job Description may require:

- AWS
- Docker
- Kubernetes

But a resume may mention:

- Amazon Web Services
- Containerized Deployments
- EC2 Infrastructure

Traditional keyword systems fail to identify these semantic relationships.

The platform solves this problem using vector embeddings and semantic retrieval.

3. Product Objectives

The system was designed with the following objectives:

- Automate resume-to-JD analysis
 - Improve semantic skill matching accuracy
 - Identify candidate skill gaps
 - Generate personalized interview preparation
 - Reduce hallucinated AI responses
 - Create ATS-compatible downloadable reports
 - Preserve contextual accuracy during document parsing
-

4. Core Features

4.1 Resume Upload & Parsing

Users can upload resumes in PDF format.

The backend automatically extracts and structures the document using:

- LangChain
- UnstructuredPDFLoader
- mode="elements"

The parser separates:

- Headers
- Titles
- Lists
- Narrative text
- Experience sections
- Skills sections

This preserves document hierarchy before chunking.

4.2 Header-Aware Chunking

Instead of arbitrary character splitting, the platform performs section-based chunking.

Example:

Traditional Chunking (Incorrect)

```
Worked on AWS deployments...  
Bachelor of Engineering in Computer Science
```

This mixes unrelated sections.

Implemented Approach

```
Header: Work Experience  
  
- Managed AWS EC2 infrastructure  
- Built Docker-based deployments  
- Developed REST APIs using Node.js
```

This preserves semantic meaning during vector retrieval.

4.3 Semantic Embedding & Vector Storage

Each structured chunk is converted into vector embeddings.

The embeddings are indexed into:

- FAISS
- ChromaDB

This enables:

- Semantic retrieval
 - Fast similarity search
 - Context-aware matching
 - Skill-based querying
-

4.4 Job Description Skill Extraction

Instead of directly embedding the entire Job Description, the system extracts only mandatory requirements.

Gemini 1.5 Flash processes the JD using constrained prompts.

Example Output:

```
[  
  "React.js",  
  "Node.js",  
  "AWS",  
  "Docker",  
  "MongoDB"  
]
```

This structured extraction improves retrieval precision.

4.5 Semantic Skill Matching

The extracted skills are used as semantic search queries against the resume vector database.

The system identifies:

- Matched skills
- Missing skills
- Weak matches
- Related technologies

Example:

JD Skill	Resume Content	Match Result
AWS	Amazon Web Services	✓ Match
Docker	Containerized deployments	✓ Match
Kubernetes	No related context found	✗ Missing

4.6 Personalized Interview Preparation

After identifying skill gaps, the platform generates:

- Personalized interview questions
- Learning recommendations
- Preparation roadmaps
- Technology-specific guidance

The generation step is context-bounded using retrieved data to reduce hallucination.

4.7 ATS-Compatible PDF Report Generation

The final analysis report is rendered using Puppeteer.

The generated PDF contains:

- Matched Skills
- Missing Skills
- Similarity Scores
- Interview Questions
- Learning Recommendations
- Candidate Summary

The exported report maintains:

- ATS compatibility
 - Consistent formatting
 - Cross-device rendering stability
-

5. System Architecture

Frontend

- React.js
- Responsive Dashboard UI
- Resume Upload Interface
- Skill Visualization Components

Backend

- Node.js
- Express.js

- LangChain Orchestration
- API Routing
- PDF Processing Pipeline

AI & Retrieval

- Gemini 1.5 Flash
- Embedding Generation
- Semantic Retrieval
- Prompt Engineering

Vector Databases

- FAISS
- ChromaDB

Rendering Engine

- Puppeteer
-

6. End-to-End Workflow

Step 1 — Resume Upload

The user uploads:

- Resume PDF
- Job Description

The frontend sends the request to the Express backend.

Step 2 — Document Parsing

The backend processes the resume using:

```
UnstructuredPDFLoader(mode="elements")
```

The document is divided into structural elements.

Step 3 — Data Sanitization

The pipeline removes:

- Extra whitespace
- Formatting artifacts
- Invalid characters
- Hidden PDF noise

This improves embedding quality.

Step 4 — Header-Aware Chunking

Chunks are grouped by semantic headers.

Example:

- Skills
 - Work Experience
 - Certifications
 - Education
-

Step 5 — Embedding Generation

Each chunk is converted into vector embeddings.

Example:

```
"AWS Deployment"  
→ [0.183, -0.928, 0.441, ...]
```

Step 6 — Vector Indexing

The embeddings are stored in:

- FAISS
 - ChromaDB
-

Step 7 — JD Skill Extraction

Gemini extracts mandatory skills from the Job Description.

Example:

```
[  
  "React.js",  
  "Docker",  
  "AWS"  
]
```

Step 8 — Semantic Retrieval

For every extracted skill:

- Vector similarity search is executed
- Top-K retrieval is applied
- Similarity scores are analyzed

Step 9 — Skill Gap Identification

Skills below the configured similarity threshold are marked as missing.

Example:

```
Kubernetes → Missing
```

Step 10 — AI-Based Interview Generation

Gemini generates:

- Mock interview questions
- Preparation guidance
- Learning suggestions

Generation is bounded by retrieved context.

Step 11 — PDF Report Export

Puppeteer renders a professional ATS-compatible report.

The report is returned to the frontend.

7. Engineering Optimizations

Top-K Retrieval

The retriever fetches only the top 5 relevant chunks.

Benefits:

- Reduces hallucination
 - Removes noisy context
 - Reduces token usage
 - Improves response quality
-

Context Filtering

Only highly relevant resume chunks are passed to the LLM.

This prevents unsupported AI generation.

Similarity Thresholding

Skills below the semantic similarity threshold are marked as missing.

This improves retrieval reliability.

Structured Prompting

Gemini is constrained using deterministic prompts.

This improves:

- JSON consistency
- Extraction reliability

- Output formatting
-

8. Why Gemini Was Used

Gemini 1.5 Flash was selected because of:

- Fast inference speed
- Low latency
- Reliable structured output generation
- Strong JSON formatting capabilities
- Cost-effective API usage
- Efficient context handling

The model was primarily used for:

- Skill extraction
 - Context-bounded generation
 - Interview question generation
 - Recommendation workflows
-

9. Security & Privacy Considerations

The system was designed with privacy-focused processing.

Implemented considerations:

- Dynamic per-session processing
 - Reduced raw data exposure
 - Context-limited prompt generation
 - Vector-based retrieval instead of full document prompts
 - Temporary processing pipelines
-

10. Performance Improvements

Optimization	Impact
Header-Aware Chunking	Improved semantic context retention
Top-K Retrieval	Reduced irrelevant context
Context Filtering	Reduced hallucinated responses

Optimization	Impact
Semantic Matching	Improved skill detection accuracy
Structured Extraction	Improved JD parsing reliability

11. Challenges Solved

Challenge 1 — PDF Structure Loss

Traditional PDF extraction flattened resumes into a single block of text.

Solution

Used `UnstructuredPDFLoader(mode="elements")` to preserve document hierarchy.

Challenge 2 — Poor Keyword Matching

Traditional systems failed to match semantically related technologies.

Solution

Implemented semantic vector retrieval.

Challenge 3 — Hallucinated AI Outputs

LLMs generated unsupported recommendations.

Solution

Applied:

- Top-K retrieval
 - Context filtering
 - Structured prompts
 - Fallback handling
-

12. Future Improvements

Planned enhancements include:

- Multi-resume ranking
 - Recruiter dashboard
 - Skill trend analytics
 - Multi-language resume support
 - Real-time interview simulation
 - Voice-based mock interviews
 - Enterprise ATS integration
-

13. Conclusion

The AI-Powered Resume Analyzer & Interview Preparation Platform demonstrates how Retrieval-Augmented Generation (RAG), semantic retrieval, structured parsing, and vector databases can be combined to build reliable AI-powered career guidance systems.

By preserving document structure, applying semantic matching, and constraining LLM generation using retrieved context, the platform improves both retrieval quality and response reliability while reducing generic AI outputs.

The architecture was intentionally designed to balance:

- Scalability
- Retrieval precision
- Low-latency generation
- ATS compatibility
- Cost optimization
- AI reliability

making it suitable for real-world resume analysis and interview preparation workflows.